

内存管理：连续分配、非连续分配与动态分区分配算法

内存管理：连续分配、非连续分配与动态分区分配算法

目标：讲清楚内存的连续分配和非连续分配，重点掌握固定分区、动态分区、分页、分段、段页式之间的关系，并系统总结首次适应、循环首次适应、最佳适应、最坏适应、邻近适应、快速适应、伙伴系统等动态分区分配算法。

0. 先把问题放到全局来看

内存分配要解决的问题是：当一个进程要运行时，操作系统应该把它放到主存的哪里？
如果从“进程是否必须占用一整块连续物理内存”来看，内存分配方式可以分成两大类：

大类	基本思想	代表方式	是否要求物理连续	主要碎片
连续分配	一个进程或一个分区占用一段连续物理内存	单一连续分配、固定分区、动态分区	要求连续	固定分区有内部碎片，动态分区有外部碎片
非连续分配	一个进程可以被拆开放到多个不连续物理位置	分页、分段、段页式	不要求整个进程连续	分页有内部碎片，分段可能有外部碎片，段页式主要有内部碎片

一句话：连续分配简单，但怕找不到足够大的连续空闲区；非连续分配复杂，但能更充分利用零散内存。

1. 什么是连续分配

1.1 连续分配的定义

连续分配是指：一个进程装入内存时，必须占用一段连续的物理地址空间。
例如某进程需要 100 KB，如果它被装入物理地址 400 KB 开始的位置，那么它占用的内存范围是：
 $[400KB, 500KB)$

中间不能断开，不能一半在 400 KB，一半在 900 KB。
连续分配的典型地址变换很简单。若进程逻辑地址为 A ，重定位寄存器保存起始物理地址 B ，则物理地址为：

$$E = B + A$$

同时还要用界限寄存器检查是否越界。若进程长度为 L ，则必须满足：

$$0 \leq A < L$$

1.2 连续分配的共同特点

连续分配的优点：

1. 地址变换简单，只需要基址加偏移。
2. 管理结构简单，适合早期操作系统。
3. 对硬件要求低。

连续分配的缺点：

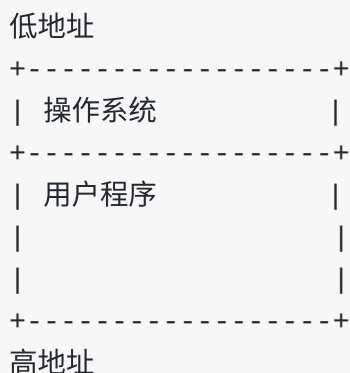
1. 一个进程必须找到足够大的连续空闲区。
2. 容易产生碎片。
3. 内存利用率不高。
4. 进程扩展困难，例如堆或栈增长时可能没有相邻空间。

2. 单一连续分配

2.1 基本思想

单一连续分配是最简单的方式：内存中通常只运行一个用户进程，操作系统占用一部分内存，剩余部分全部给用户程序。

示意：



2.2 特点

优点是简单，几乎不需要复杂的内存管理。

缺点也很明显：

1. 同一时刻只能运行一个用户程序。
2. 内存利用率低。
3. 不支持现代意义上的多道程序设计。

它适合早期单道批处理系统，不适合现代操作系统。

3. 固定分区分配

3.1 基本思想

固定分区分配是把内存预先划分成若干个固定大小的分区。每个分区一次只能装入一个进程。

示意：

低地址



高地址

进程来了以后，操作系统选择一个能够容纳它的空闲分区。

3.2 分区大小可以相等，也可以不等

固定分区有两种做法：

类型	特点	问题
分区大小相等	管理简单	大程序放不下，小程序浪费多
分区大小不等	能适应不同大小进程	管理稍复杂，仍有浪费

3.3 固定分区的碎片

固定分区的主要问题是内部碎片。

例如某分区大小为 100 KB，进程只需要 70 KB，那么剩下 30 KB 不能再分给其他进程，只能浪费在该分区内部。

这部分浪费就是内部碎片：

$$\text{内部碎片} = \text{分区大小} - \text{进程大小}$$

3.4 固定分区的优缺点

优点：

1. 比单一连续分配支持更多并发进程。
2. 管理简单。
3. 地址变换仍然比较简单。

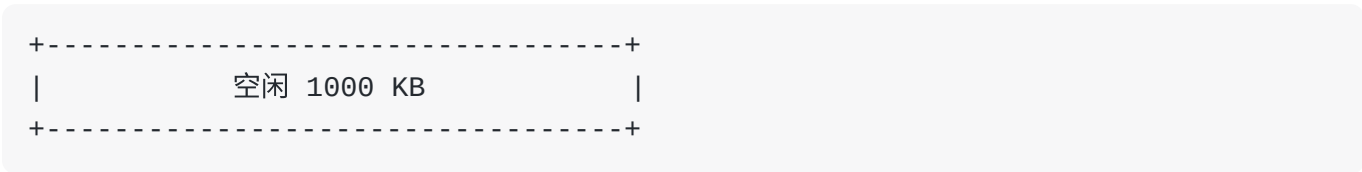
缺点：

1. 分区数固定，限制并发进程数。
2. 分区大小固定，容易产生内部碎片。
3. 大进程可能因为没有足够大的固定分区而无法装入。

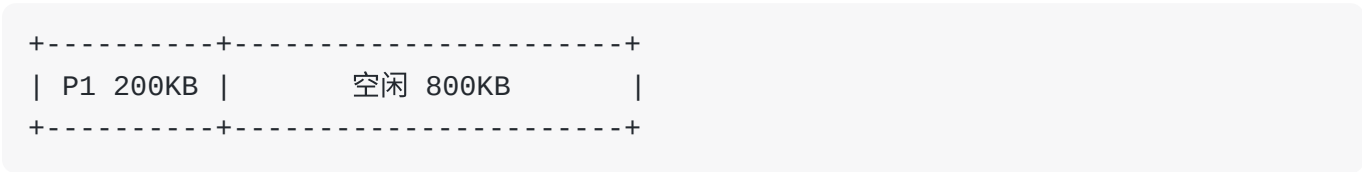
4. 动态分区分配

4.1 基本思想

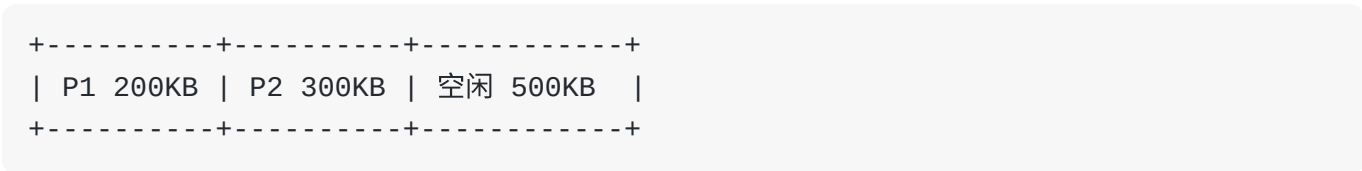
动态分区分配不预先划分固定分区，而是在进程装入时，根据进程大小动态划出一块连续内存。
例如初始内存有一个大空闲区：



来了一个 200 KB 的进程，就切出 200 KB：



再来一个 300 KB 的进程：



当进程退出时，它占用的空间重新变为空闲区。

4.2 动态分区需要维护什么信息

操作系统通常需要维护空闲分区表或空闲分区链。
空闲分区表可以写成：

空闲区编号	起始地址	大小
1	100 KB	60 KB
2	300 KB	180 KB
3	700 KB	400 KB

每次分配内存时，就在这些空闲区里找一个合适的。

4.3 动态分区的地址变换

动态分区仍然属于连续分配，所以地址变换仍然是基址加偏移。
若进程被装入物理起始地址 B ，逻辑地址为 A ，则：

$$E = B + A$$

还要检查：

$$A < L$$

其中 L 是进程占用分区的长度。

4.4 动态分区的碎片

动态分区主要产生外部碎片。

例如内存中有三个空闲区：

空闲 10 KB、空闲 20 KB、空闲 30 KB

总空闲空间为：

$$10 + 20 + 30 = 60KB$$

但如果某进程需要 40 KB 连续空间，就无法装入。因为没有任何一个单独空闲区达到 40 KB。

这就是外部碎片：空闲空间总量足够，但不连续，导致无法使用。

4.5 紧凑技术

为了解决外部碎片，可以进行紧凑，也叫拼接或内存压缩。

它把已分配的进程移动到一端，把零散空闲区合并成一个大空闲区。

紧凑前：

+---+---+---+---+---+
| P1 | 空闲 | P2 | 空闲 | P3 |
+---+---+---+---+---+

紧凑后：

+---+---+---+---+---+
| P1 | P2 | P3 | 空闲 |
+---+---+---+---+---+

缺点是代价大，因为要移动进程，还要修改相关地址映射信息。

5. 动态分区分配算法总览

动态分区分配算法要解决的问题是：面对多个空闲分区，应选择哪一个分给新进程？

常见算法如下：

算法	英文	基本思想	空闲区通常如何组织
首次适应	First Fit	从头找第一个足够大的空闲区	按地址递增排序
循环首次适应/邻近适应	Next Fit	从上次查找结束位置继续找	按地址递增的循环链
最佳适应	Best Fit	找能容纳进程的最小空闲区	按容量递增排序或全表扫描
最坏适应	Worst Fit	找最大的空闲区	按容量递减排序或全表扫描

算法	英文	基本思想	空闲区通常如何组织
快速适应	Quick Fit	为常见大小维护多个空闲链表	按规格分类链表
伙伴系统	Buddy System	按 2^k 大小分裂和合并	多个 2^k 空闲链表

6. 首次适应算法

6.1 基本思想

首次适应算法从空闲分区表或空闲链表的表头开始查找，找到第一个大小不小于请求大小的空闲区，就把它分配出去。

若进程请求大小为 S ，某空闲区大小为 H ，只要满足：

$$H \geq S$$

就可以分配。

如果 $H > S$ ，剩余部分继续作为空闲区。

6.2 示例

空闲区按地址从低到高排列：

编号	起始地址	大小
1	100	20 KB
2	200	50 KB
3	400	100 KB
4	700	30 KB

现在进程请求 45 KB。

首次适应算法从表头开始找：

1. 20 KB 不够。
2. 50 KB 够。
3. 选择 50 KB 这个空闲区。

分配后剩余：

$$50\text{KB} - 45\text{KB} = 5\text{KB}$$

这个 5 KB 继续留作空闲区。

6.3 优缺点

优点：

1. 查找速度通常较快。
2. 实现简单。
3. 大空闲区倾向于保留在高地址部分。

缺点：

1. 低地址部分会逐渐产生很多小碎片。
2. 每次都从头查找，表头附近可能越来越拥挤。

6.4 适合记忆

首次适应的关键词是：

从头找，第一个够用就分配。

7. 循环首次适应算法 / 邻近适应算法

7.1 基本思想

循环首次适应也叫邻近适应。它不是每次都从表头开始，而是从上一次找到空闲区的位置之后继续查找。

如果查到表尾还没有找到，就绕回表头继续查找。

它的核心是维护一个查找指针。

上次分配位置 → 从这里继续往后找 → 到表尾后回到表头

7.2 示例

空闲区按地址排列：

编号	起始地址	大小
1	100	20 KB
2	200	50 KB
3	400	100 KB
4	700	30 KB

假设上次查找停在编号 2 后面，现在请求 25 KB，则从编号 3 开始找：

1. 编号 3：100 KB，够用。
2. 直接选择编号 3。

它不会从编号 1 重新开始。

7.3 优缺点

优点：

1. 避免每次都从低地址开始查找。
2. 空闲区使用更均匀。
3. 查找开销可能小于首次适应。

缺点：

1. 可能破坏高地址的大空闲区。
2. 大空闲区不容易长期保留。
3. 整体效果不一定优于首次适应。

7.4 适合记忆

循环首次适应的关键词是：

从上次位置继续找，找到第一个够用的。

8. 最佳适应算法

8.1 基本思想

最佳适应算法选择所有能满足请求的空闲区中最小的那个。

也就是找满足 $H \geq S$ 的空闲区中 H 最小者。

公式化表示：

$$H^* = \min\{H \mid H \geq S\}$$

它的想法是：尽量别浪费大空闲区。

8.2 示例

空闲区如下：

编号	大小
1	20 KB
2	50 KB
3	100 KB
4	30 KB

请求 25 KB。

能满足的有 50 KB、100 KB、30 KB，其中最小的是 30 KB，所以选择 30 KB。

分配后剩余：

$$30\text{KB} - 25\text{KB} = 5\text{KB}$$

8.3 优缺点

优点：

1. 每次局部浪费看起来最少。
2. 能尽量保留大的空闲区。

缺点：

1. 容易产生大量很小、难以再利用的碎片。

2. 如果空闲区不按大小排序，查找需要扫描整个表。
3. 即使按大小排序，分配和回收后也要维护顺序。

8.4 为什么最佳适应不一定最佳

“最佳适应”只是对当前这一次分配最省空间，不代表全局最优。

例如经常把 30 KB 分区分给 25 KB 进程，就会留下很多 5 KB 小碎片。这些碎片可能长期无法利用。

所以最佳适应的典型问题是：**小碎片太多**。

8.5 适合记忆

最佳适应的关键词是：

找最小的够用空闲区。

9. 最坏适应算法

9.1 基本思想

最坏适应算法和最佳适应相反。它总是选择当前最大的空闲区。

如果请求大小为 S ，它选择最大的 H ，且满足：

$$H \geq S$$

它的想法是：从最大空闲区里切一块出去后，剩下的部分仍然可能比较大，还能继续使用。

9.2 示例

空闲区如下：

编号	大小
1	20 KB
2	50 KB
3	100 KB
4	30 KB

请求 25 KB。

最坏适应选择最大的 100 KB 空闲区。分配后剩余：

$$100\text{KB} - 25\text{KB} = 75\text{KB}$$

9.3 优缺点

优点：

1. 不容易留下特别小的碎片。
2. 剩余空闲区通常还比较大。

缺点：

1. 会很快消耗大空闲区。
2. 当大进程到来时，可能没有足够大的空闲区。
3. 整体内存利用效果通常不理想。

9.4 适合记忆

最坏适应的关键词是：

找最大的空闲区来切。

10. 快速适应算法

10.1 基本思想

快速适应算法把常见大小的空闲分区单独组织成多个链表。例如：

8 KB 空闲链表
16 KB 空闲链表
32 KB 空闲链表
64 KB 空闲链表
128 KB 空闲链表

请求某种常见大小时，直接到对应链表取一个分区，不需要反复扫描整个空闲表。

10.2 优点

1. 分配速度很快。
2. 适合请求大小集中在若干规格的场景。
3. 减少查找时间。

10.3 缺点

1. 回收时合并相邻空闲区比较复杂。
2. 如果规格设计不好，会浪费空间。
3. 对不常见大小的请求不一定友好。

10.4 适合记忆

快速适应的关键词是：

按大小分类，直接从对应链表取。

11. 伙伴系统

11.1 基本思想

伙伴系统把内存按 2^k 大小进行管理。请求内存时，系统分配不小于请求大小的最小 2^k 块。

例如请求 70 KB，如果块大小只能是 64 KB、128 KB、256 KB，那么会分配 128 KB。

11.2 分裂过程

如果没有刚好大小的空闲块，就把更大的块不断对半分裂。

例如有一个 512 KB 空闲块，请求 100 KB：

```
512 KB
  ↓ 分裂
256 KB + 256 KB
  ↓ 继续分裂其中一个 256 KB
128 KB + 128 KB + 256 KB
  ↓
分配一个 128 KB 给进程
```

11.3 合并过程

当一个块释放时，如果它的伙伴块也空闲，就可以合并成更大的块。

例如两个相邻的 128 KB 伙伴块都空闲，就可以合并为 256 KB。

再看它的 256 KB 伙伴是否空闲，若空闲还能继续合并。

11.4 伙伴的含义

伙伴不是任意相邻块，而是由同一个大块分裂出来的两个大小相同的块。

例如一个 256 KB 块分裂成两个 128 KB 块，这两个 128 KB 块互为伙伴。

11.5 优缺点

优点：

1. 分配和回收较快。
2. 合并规则清晰。
3. 比普通动态分区更容易管理。

缺点：

1. 由于按 2^k 分配，可能产生内部碎片。
2. 请求大小稍大于 2^{k-1} 时，会分配 2^k ，浪费较明显。

例如请求 129 KB，只能分配 256 KB，内部浪费为：

$$256\text{KB} - 129\text{KB} = 127\text{KB}$$

11.6 适合记忆

伙伴系统的关键词是：

不够就对半分裂，释放后找伙伴合并。

12. 非连续分配

12.1 非连续分配的定义

非连续分配是指：一个进程不必占用一整段连续物理内存，而是可以拆成多个部分，分别放到不同的物理位置。

最典型的是分页：

```
进程逻辑页： 页0  页1  页2  页3
物理页框：   框8  框2  框15 框4
```

进程逻辑上连续，物理上不连续。

12.2 非连续分配为什么重要

连续分配的问题是必须找大连续空闲区。如果内存碎成很多小块，大进程可能无法装入。
非连续分配则可以把进程拆开，塞进多个小空闲区，从而提高内存利用率。

12.3 非连续分配的代价

非连续分配需要额外的数据结构来记录映射关系。

方式	需要的数据结构	地址变换
分页	页表	页号查页框号
分段	段表	段号查基址和段长
段页式	段表 + 页表	先查段表，再查页表

非连续分配的地址变换更复杂，因此通常需要硬件支持，例如 MMU 和 TLB。

13. 分页、分段、段页式与连续/非连续的关系

13.1 分页

分页把逻辑地址空间分成固定大小的页，把物理内存分成同样大小的页框。
进程的不同页可以放入不同页框，所以分页属于非连续分配。

分页地址结构：

```
| 页号 p | 页内偏移 d |
```

地址变换：

$$E = fL + d$$

其中 f 是页框号， L 是页大小。

分页特点：

- 1. 无外部碎片。

2. 有内部碎片。
3. 对用户透明。
4. 需要页表。

13.2 分段

分段按程序逻辑模块划分，例如代码段、数据段、栈段。

每个段内部通常连续存放，但不同段之间可以不连续。因此从整个进程角度看，分段是一种非连续分配；但从单个段角度看，每个段仍要求连续空间。

分段地址结构：

| 段号 s | 段内偏移 d |

地址变换：

$$E = b_s + d$$

其中 b_s 是段基址。

分段特点：

1. 符合程序逻辑。
2. 方便共享和保护。
3. 段长可变。
4. 会产生外部碎片。

13.3 段页式

段页式先分段，再把每个段分页。它既保留了分段的逻辑结构，又通过分页实现离散装入。

段页式地址结构：

| 段号 s | 段内页号 p | 页内偏移 d |

地址变换：

段号 $s \rightarrow$ 查段表 $\rightarrow$ 找到该段页表
段内页号 $p \rightarrow$ 查页表 $\rightarrow$ 得到页框号 f
物理地址 = $f \times \text{页大小} + d$

段页式特点：

1. 以段为逻辑保护单位。
2. 以页为物理分配单位。
3. 无外部碎片。
4. 有内部碎片。
5. 表结构和地址变换更复杂。

14. 连续分配与非连续分配的系统对比

对比项	连续分配	非连续分配
物理存放	一个进程或分区占连续空间	可分散到多个位置
地址变换	基址 + 偏移	查表映射
硬件支持	较少	通常需要 MMU/TLB
内存利用率	较低	较高
碎片	固定分区有内部碎片，动态分区有外部碎片	分页有内部碎片，分段有外部碎片，段页式主要有内部碎片
共享保护	较弱	更灵活
实现复杂度	简单	复杂
代表方式	单一连续、固定分区、动态分区	分页、分段、段页式

15. 动态分区算法对比表

算法	选择策略	优点	缺点	典型碎片趋势
首次适应	第一个够用的	简单、速度较快	低地址小碎片多	表头附近碎片多
循环首次适应	从上次位置继续找第一个够用的	分配更均匀	可能破坏大空闲区	分布较均匀
最佳适应	最小的够用空闲区	单次浪费小	容易留下很多极小碎片	小碎片多
最坏适应	最大空闲区	剩余块仍较大	大空闲区消耗快	大块减少快
快速适应	按规格链表快速取	分配快	合并复杂，规格依赖强	取决于规格设计
伙伴系统	按 2^k 分裂合并	分裂合并规则清晰	可能有内部碎片	2^k 对齐浪费

16. 做题常见套路

16.1 判断属于哪类分配

看到“一个进程必须装入连续内存区域”，通常是连续分配。

看到“页、页框、页表”，通常是分页，属于非连续分配。

看到“段号、段长、段基址”，通常是分段。

看到“段表指出页表始址”，通常是段页式。

16.2 判断碎片类型

记忆规则：

固定大小 → 容易内部碎片

可变大小且要求连续 → 容易外部碎片

分页 → 内部碎片

动态分区/分段 → 外部碎片

段页式 → 内部碎片

16.3 动态分区算法选择题

遇到空闲分区表和请求大小，按算法逐个判断：

- 1. 首次适应：按表中顺序找第一个够用的。
- 2. 循环首次适应：从上次位置后面开始找第一个够用的。
- 3. 最佳适应：在所有够用的里面找最小的。
- 4. 最坏适应：在所有够用的里面找最大的。

16.4 动态分区分配后如何更新空闲表

若空闲区大小为 H ，请求大小为 S ：

- 若 $H = S$ ，该空闲区从表中删除。
- 若 $H > S$ ，切出 S ，剩余 $H - S$ 继续留在空闲表。

公式：

$$\text{剩余空闲区大小} = H - S$$

若按地址递增排序，还要更新剩余空闲区的起始地址。

16.5 回收时如何合并

回收一个分区时，要看它前后是否有相邻空闲区。

情况有四种：

情况	操作
前后都不是空闲区	新增一个空闲区
前面是空闲区，后面不是	与前面的空闲区合并
前面不是，后面是空闲区	与后面的空闲区合并
前后都是空闲区	三者合并成一个大空闲区

合并依据是物理地址连续，而不是表中相邻。

17. 综合例题：动态分区算法

空闲区表如下，按地址递增排列：

空闲区	起始地址	大小
A	100	20 KB
B	200	80 KB
C	400	50 KB
D	600	120 KB

现在请求 45 KB。

17.1 首次适应

从头找：

- A：20 KB，不够。
- B：80 KB，够。

所以选择 B，剩余：

$$80 - 45 = 35\text{KB}$$

17.2 最佳适应

能满足 45 KB 的空闲区有：

B：80 KB
C：50 KB
D：120 KB

最小的是 C：50 KB，所以选择 C，剩余：

$$50 - 45 = 5\text{KB}$$

17.3 最坏适应

最大的空闲区是 D：120 KB，所以选择 D，剩余：

$$120 - 45 = 75\text{KB}$$

17.4 循环首次适应

如果上次查找停在 B 后面，则从 C 开始：

- C：50 KB，够。

所以选择 C。

如果上次查找停在 C 后面，则从 D 开始，选择 D。

循环首次适应的结果依赖“上次查找指针”的位置。

18. 最容易混的点

18.1 动态分区和分段有什么区别？

动态分区是一种物理内存分配策略：进程来了，就按进程大小切一块连续内存。

分段是一种按程序逻辑划分地址空间的管理方式：程序被分为代码段、数据段、栈段等。

它们都可能使用可变长度连续区域，所以都可能产生外部碎片，但关注点不同。

18.2 分段是不是非连续分配？

从整个进程看，分段可以是非连续的，因为不同段可以放在不同位置。

但从单个段看，每个段内部通常连续存放，所以分段仍然会产生外部碎片。

18.3 动态分区有没有内部碎片？

严格按需分配时，动态分区主要没有内部碎片，主要问题是外部碎片。

但实际系统可能因为对齐、最小分配单位等产生少量内部浪费。408 语境下通常记：动态分区产生外部碎片。

18.4 最佳适应为什么会产生很多小碎片？

因为它总是找最接近请求大小的空闲区，切完后剩余部分往往很小。这些小碎片可能不能满足后续请求。

18.5 最坏适应为什么不一定好？

它总是切最大的空闲区，会快速破坏大块连续空间。后面如果来大进程，反而可能没有足够大的空闲区。

19. 一页速记

1. 连续分配：进程占一段连续物理内存，地址变换是基址加偏移。
2. 非连续分配：进程可分散存放，需要页表、段表等映射结构。
3. 单一连续分配：一次通常只运行一个用户进程，简单但利用率低。
4. 固定分区：预先切分区，一个分区一个进程，主要有内部碎片。
5. 动态分区：按进程大小动态切分，主要有外部碎片。
6. 外部碎片：总空闲够，但没有足够大的连续空闲区。
7. 内部碎片：分配给进程的空间内部有没用完的部分。
8. 首次适应：从头找第一个够用的。
9. 循环首次适应：从上次位置继续找第一个够用的。
10. 最佳适应：找最小的够用空闲区，容易留下小碎片。
11. 最坏适应：找最大的空闲区，容易消耗大空闲区。
12. 快速适应：按常见大小分类链表，分配快但合并复杂。
13. 伙伴系统：按 2^k 分裂和合并，规则清晰但可能有内部碎片。
14. 分页：非连续分配，无外部碎片，有内部碎片。
15. 分段：逻辑友好，段间可不连续，但段内连续，可能有外部碎片。
16. 段页式：先分段再分页，保留逻辑结构，同时用页框离散分配。